Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №52

ТЕМА: Динамічна генерація віджетів у циклі та створення списків/карток даних у CTkScrollableFrame.

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- навчитися динамічно створювати віджети в циклі на основі даних;
- ознайомитися з прийомом побудови списку «карток» даних у CTkScrollableFrame;
- вивчити спосіб зв'язування кожної динамічно створеної кнопки з власними, індивідуальними даними;
- розвинути навички побудови інтерфейсів, вміст яких залежить від структури даних, а не «зашитий» у код.

Теоретичні відомості

1. Навіщо потрібна динамічна генерація віджетів

Досі в усіх прикладах кількість віджетів у програмі була наперед відомою й фіксованою — ми писали окремий рядок коду для кожної кнопки чи мітки. Але в реальних застосунках кількість елементів часто залежить від даних, які заздалегідь невідомі: список товарів у магазині, список повідомлень чату, результати пошуку. У таких випадках віджети створюють не вручну, а в циклі — по одному на кожен елемент вихідних даних (списку чи словника).

2. Створення міток у циклі за списком

Найпростіший приклад динамічної генерації — вивести список рядків, кожен окремим CTkLabel, у циклі for:

```
products = ['Зошит', 'Ручка', 'Лінійка', 'Пенал', 'Фарби']

for product in products:
    ctk.CTkLabel(app, text=product).pack(pady=5)
```

Головна ідея тут проста: кількість викликів CTkLabel(...) визначається не жорстко заданим числом у коді, а фактичною довжиною списку products — якщо список міститиме 3 елементи чи 300, код циклу залишається абсолютно тим самим.

3. Побудова «карток» даних у CTkScrollableFrame

Реальні інтерфейси рідко обмежуються однією міткою на елемент — зазвичай кожен елемент списку показують у вигляді невеликої «картки»:

окремого CTkFrame, що містить кілька віджетів (наприклад, назву й ціну товару), розташованих поруч:

```
products = [
    ('Зошит', '25 грн'), ('Ручка', '12 грн'), ('Лінійка', '8 грн'),
    ('Пенал', '150 грн'), ('Фарби', '95 грн')
]

scroll_frame = ctk.CTkScrollableFrame(app, width=340, height=220,
corner_radius=12)
scroll_frame.pack(padx=15, pady=15, fill='both', expand=True)

for name, price in products:
    card = ctk.CTkFrame(scroll_frame, corner_radius=10)
    card.pack(fill='x', pady=5, padx=5)
    ctk.CTkLabel(card, text=name).pack(side='left', padx=12,
pady=10)
    ctk.CTkLabel(card, text=price,
text_color='#5aa9e6').pack(side='right', padx=12)
```

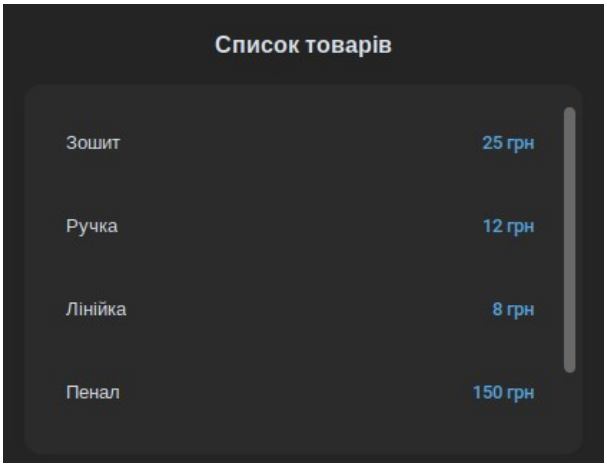


Рис. 1. Список товарів, побудований динамічно в циклі — кожна «картка» це окремий CTkFrame із двома мітками всередині

Такий підхід («один елемент даних → одна картка з кількома віджетами всередині») є одним із найпоширеніших патернів побудови інтерфейсів у реальних застосунках: списки контактів, стрічки повідомлень, результати пошуку товарів — усе це, по суті, картки, згенеровані цим самим способом.

4. Додавання кнопки дії до кожної картки

Часто кожна картка потребує власної кнопки дії (наприклад, «Видалити» чи «Детальніше»), яка має «пам'ятати», до якого саме елемента списку вона належить. Найпоширеніша помилка тут — використати змінну циклу безпосередньо в лямбда-функції: усі кнопки в результаті посилатимуться на останнє значення цієї змінної. Розв'язання — «захопити» поточне значення як значення параметра за замовчуванням лямбда-функції:

```
def delete_item(item_name):
    print('Видалити:', item_name)

for name, price in products:
    card = ctk.CTkFrame(scroll_frame, corner_radius=10)
    card.pack(fill='x', pady=5, padx=5)
    ctk.CTkLabel(card, text=f'{name} — {price}').pack(side='left',
padx=12)
    ctk.CTkButton(
        card, text='Видалити', width=90,
        command=lambda n=name: delete_item(n)
    ).pack(side='right', padx=12)
```

Запис lambda n=name: delete_item(n) створює для кожної ітерації циклу окрему лямбда-функцію зі своїм власним значенням параметра n, зафіксованим саме на момент створення кнопки, — тому кожна кнопка коректно «пам'ятає» саме свій товар, а не останній елемент списку.

5. Очищення і повторна побудова динамічного вмісту

Іноді потрібно повністю перебудувати список карток заново (наприклад, після фільтрації чи сортування даних). Для цього спочатку видаляють усі наявні дочірні віджети контейнера, а потім заново виконують цикл створення карток:

```
def refresh_list(data):
    for widget in scroll_frame.winfo_children():
        widget.destroy()
    for name, price in data:
        card = ctk.CTkFrame(scroll_frame, corner_radius=10)
        card.pack(fill='x', pady=5, padx=5)
        ctk.CTkLabel(card, text=f'{name} —
{price}').pack(side='left', padx=12)
```

Метод winfo_children() повертає список усіх дочірніх віджетів контейнера — саме через нього і реалізують «очищення» динамічного списку перед його повторним заповненням.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Кількість імен залежить від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
1–3	кількість імен n = 6 + N mod 3	n = 7

б) Кожне завдання — окремий файл Прізвище_ПР52_зN.py (наприклад, Ivanenko_PR52_z3.py), код текстом, а не фото чи скріншотом. Файли з

прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного вікна додайте до звіту скріншот із видимим заголовком вікна. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів (кнопок, міток, повідомлень у консоль) — точно як в умові. Значення змінної циклу в lambda передавайте через параметр за замовчуванням.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР52_з1.py. Створіть список із n імен (n за вашим варіантом) і виведіть кожне окремою CTkLabel у циклі.

   Приклад (N = 7): n = 7.

2. Перетворіть список на картки (CTkFrame) у CTkScrollableFrame: ім'я та оцінка за формулою $7 + (i \cdot N) \bmod 6$, i = 0, 1, …

   Приклад (N = 7): оцінки [7, 8, 9, 10, 11, 12, 7].

3. Додайте до кожної картки кнопку «Детальніше», що виводить у консоль «Ім'я: …, оцінка: …» саме цієї картки (захоплення значення в lambda: lambda x=item: …).

   Приклад: кнопка другої картки → «Ім'я: <друге ім'я>, оцінка: 8».

4. Напишіть функцію rebuild(data), що видаляє всі картки й будує їх заново з нового списку, і кнопку, що викликає її з іншим набором із 3 записів.

   Приклад: після кліку у списку рівно 3 картки.

5. Додайте до кожної картки кнопку «Видалити», що знищує лише цю картку (destroy() для її фрейму).

   Приклад: після видалення решта карток лишається без змін.

6. Завдання підвищеної складності*. «Список справ»: CTkEntry і кнопка «Додати» додають картку зі справою та кнопкою «Виконано»; порожній текст (після strip()) не додається; список справ зберігається також у звичайному списку Python, синхронізованому з відображенням.

Приклад: додано «Купити хліб» → картка; «Виконано» → картка зникає, зі списку Python — теж.

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, усі кнопки виводять останній елемент (не захоплено значення), видаляється весь список замість однієї картки, список Python не синхронізований) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити поведінку картки чи набір даних). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чому в реальних застосунках віджети часто створюють у циклі, а не вручну для кожного елемента?
2. З яких частин зазвичай складається «картка» даних в інтерфейсі?
3. Яка типова помилка виникає при використанні змінної циклу безпосередньо в лямбда-функції команди кнопки, і як її уникнути?
4. Який метод повертає список усіх дочірніх віджетів контейнера?
5. Як програмно видалити один конкретний віджет (наприклад, одну картку) з інтерфейсу?